MMDU MULLANA OFFICIAL SYLLABUS • BCSE-012 • UNIT 3

Unit 3: HTML5 & Modern Web Application APIs (Semantics, Canvas, Storage & Media)

Subject: Fundamentals of Web Technologies (BCSE-012) • **Exam Weightage:** 24 - 28 Marks • **Study Time:** 40 mins

HTML5 paradigm shift; 5 major limitations of HTML4; Structural semantic page layout blueprint; Native multimedia

multi-codec fallback (

1. Evolution & Advantages of HTML5 over HTML4

1. The Paradigm Shift: HTML4 Limitations vs. HTML5 Solutions

Ratified as an official W3C Recommendation in October 2014, **HTML5** modernized web development from a simple text formatting language into a robust application execution runtime.

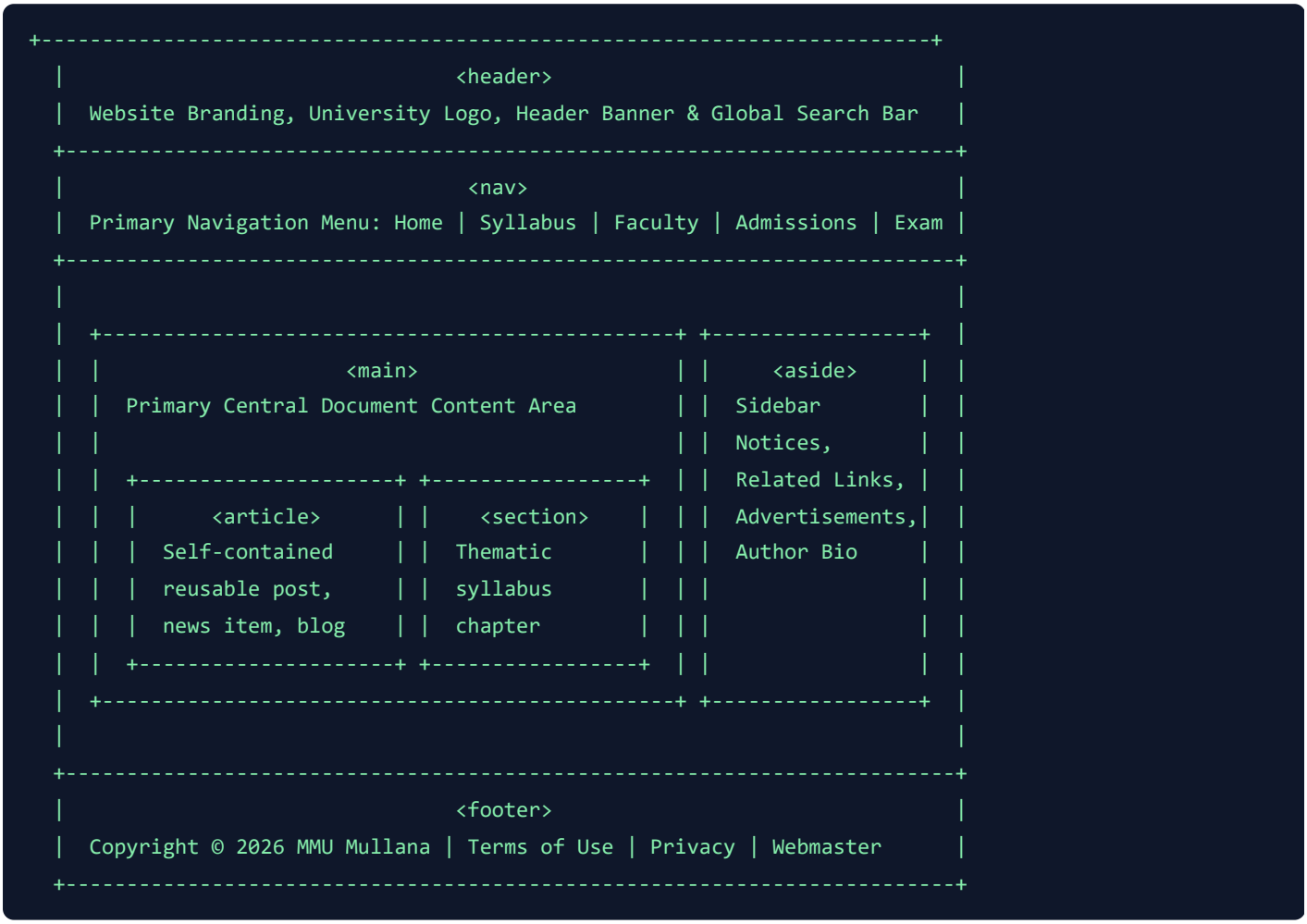
HTML4 Limitation	Problem Encountered	HTML5 Modern Solution
Lack of Semantic Structure	Heavy reliance on meaningless and generic "div-soup". Screen readers could not parse document meaning.	Native structural semantic elements (, , , , , ,).
Plugin Vulnerabilities & Crashes	Video and audio playback required proprietary browser plugins (Adobe Flash, Silverlight), causing crashes and severe security holes.	Native hardware-accelerated and tags with multi-codec fallbacks and subtitle tracks.
Inadequate Client-Side Storage	Depended solely on 4KB HTTP Cookies transmitted across the network with every HTTP request header, causing network overhead.	Web Storage API (`localStorage` and `sessionStorage`) offering 5MB–10MB isolated client-side storage.
No Native Graphics Rendering	Drawing diagrams, charts, or game animations required complex Flash applets.	High-performance scriptable 2D raster drawing via the API and native scalable vector graphics ( ).

| **Poor Mobile & Error-Handling Discipline** | HTML4 lacked standardized rules for malformed markup, causing browsers to implement inconsistent parsing workarounds. | Strict standardized parsing algorithms, mobile-first responsive design, and offline capability via Service Workers. |

2. HTML5 Structural Semantic Elements Layout Blueprint

Structural Semantic Architecture of a Modern HTML5 Webpage

HTML5 introduces structural semantic elements that explicitly describe their meaning to both browser rendering engines and screen readers:



Detailed Definitions of Semantic Tags:

- **:** Represents introductory content, branding logos, navigational search utilities, or author headings.
- **:** Encloses the major primary navigation blocks containing links to other pages or sections.
- **:** Contains the central dominant content unique to that document; must not be duplicated inside headers or footers.

- : A complete, self-contained composition that is independently distributable or reusable (e.g. blog post, news story).
- : Represents a standalone thematic grouping of content, typically with its own heading (e.g. chapters, tabs).
- : Content tangentially related to the main content (e.g. sidebars, callout boxes, related article links).
- : Houses author information, copyright notices, terms of service, and back-to-top links.

3. Native Multimedia & Modern HTML5 Form Controls

1. Native Multimedia (

HTML5 eliminates third-party plugins by embedding media directly in the DOM:

```
<!-- Native HTML5 Video Element with Multi-Codec Fallback & Subtitles -->
<video width="640" height="360" controls poster="poster_image.jpg" preload="metadata">
  <source src="lecture.mp4" type="video/mp4">
  <source src="lecture.webm" type="video/webm">
  <track src="subtitles_en.vtt" kind="subtitles" srclang="en" label="English">
  Your browser does not support native HTML5 video playback.
</video>

<!-- Native HTML5 Audio Element -->
<audio controls preload="auto">
  <source src="podcast.mp3" type="audio/mpeg">
  <source src="podcast.ogg" type="audio/ogg">
  Your browser does not support the audio element.
</audio>
```

2. Modern HTML5 Form Inputs & Attributes

HTML5 modernized forms with built-in client-side validation without requiring custom JavaScript:

```
<form action="/submit_admission" method="POST">
  <!-- Email validation with regex check -->
  <label for="email">University Email:</label>
  <input type="email" id="email" name="email" placeholder="student@mmumullana.org" required>

  <!-- Numeric bounds -->
  <label for="age">Age (17-30):</label>
  <input type="number" id="age" name="age" min="17" max="30" required>

  <!-- Slider input -->
  <label for="scale">Skill Level (1-10):</label>
  <input type="range" id="scale" name="skill" min="1" max="10" value="7">

  <!-- Native Date Picker -->
  <label for="dob">Date of Birth:</label>
  <input type="date" id="dob" name="dob">

  <!-- Datalist Autocomplete -->
  <label for="course">Choose Branch:</label>
  <input list="branchList" id="course" name="branch">
  <datalist id="branchList">
    <option value="Computer Science & Engineering">
    <option value="Information Technology">
    <option value="Mechanical Engineering">
  </datalist>

  <button type="submit">Submit Application</button>
</form>
```

4. Canvas 2D Graphics API & Web Storage (Cookies vs LocalStorage)

1. The HTML5

The tag provides an immediate-mode resolution-dependent bitmap surface scriptable via JavaScript:

```
<canvas id="barChart" width="400" height="180" style="border:1px solid #cbd5e1;"></canvas>
<script>
  const canvas = document.getElementById("barChart");
  const ctx = canvas.getContext("2d");

  // 1. Draw Rectangular Bar
  ctx.fillStyle = "#0284c7";
  ctx.fillRect(40, 50, 50, 100);

  // 2. Draw Second Bar
  ctx.fillStyle = "#10b981";
  ctx.fillRect(120, 30, 50, 120);

  // 3. Draw Circle
  ctx.beginPath();
  ctx.arc(280, 90, 40, 0, 2 * Math.PI);
  ctx.fillStyle = "#f59e0b";
  ctx.fill();
  ctx.stroke();

  // 4. Render Text Label
  ctx.font = "bold 13px Segoe UI, sans-serif";
  ctx.fillStyle = "#0f172a";
  ctx.fillText("Performance Chart", 40, 25);
</script>
```

2. Client-Side Web Storage: Cookies vs. LocalStorage vs. SessionStorage



Dimension	HTTP Cookies	LocalStorage (HTML5)	SessionStorage (HTML5)
Storage Capacity	Strictly ~4 Kilobytes	~5 to 10 Megabytes	~5 Megabytes
Data Lifetime	Configurable via `Expires` or `Max-Age`	**Permanent** (persists after browser restart)	**Session-Only** (cleared when browser tab closes)
Network Transmission	Sent automatically on every HTTP header	**Never sent to server** (Client-only)	**Never sent to server** (Client-only)
Access Scope	Domain & path scoped	Origin scoped (shared across all tabs)	Tab scoped (isolated to single window)

| **JavaScript API** | Complex string manipulation (`document.cookie`) | `setItem()` , `getItem()` , `removeItem()` | `setItem()` , `getItem()` , `removeItem()` |

5. University Solved Exam Questions (With Schematics)

University Examination Solved Questions (Unit III)

📘 Section A (1 Mark): LocalStorage vs SessionStorage Lifetimes

University Question (1 Mark):
Differentiate the persistence lifetime of localStorage vs. sessionStorage as illustrated in Figure Q3.

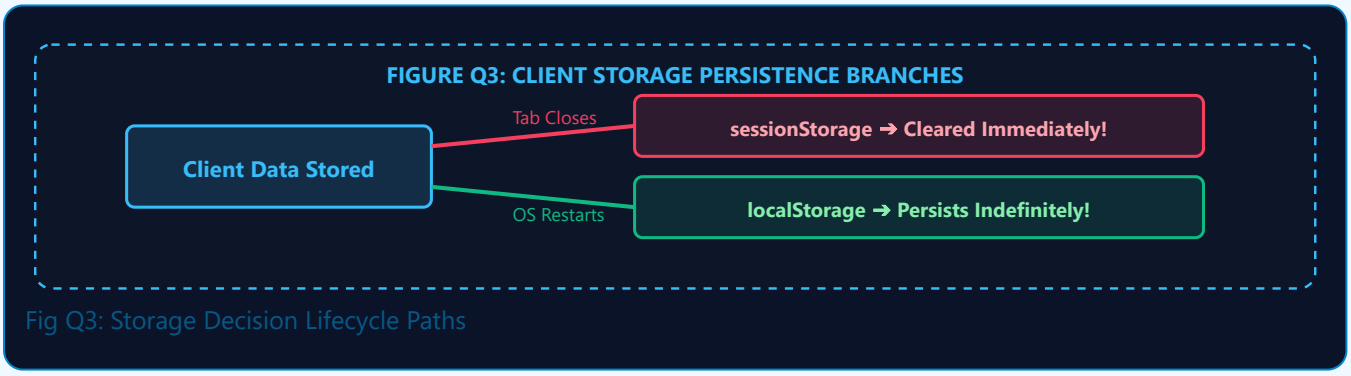


Fig Q3: Storage Decision Lifecycle Paths

Step-by-Step Solution:

- **`localStorage` :** Persists across browser sessions and computer reboots until explicitly cleared by the user or script (`localStorage.clear()`).

- **`sessionStorage` :** Confined to the lifespan of the specific browser tab; automatically purged upon tab closure.

🎯 `sessionStorage` = Tab Lifetime (Temporary) | `localStorage` = Permanent (Across Restarts).

📘 Section D (6 Marks): Dynamic Canvas Bar Chart Implementation

University Question (6 Marks):

Develop complete JavaScript and HTML5 code to draw a dynamic comparison bar chart with three subject scores on a

as shown in Figure Q15.

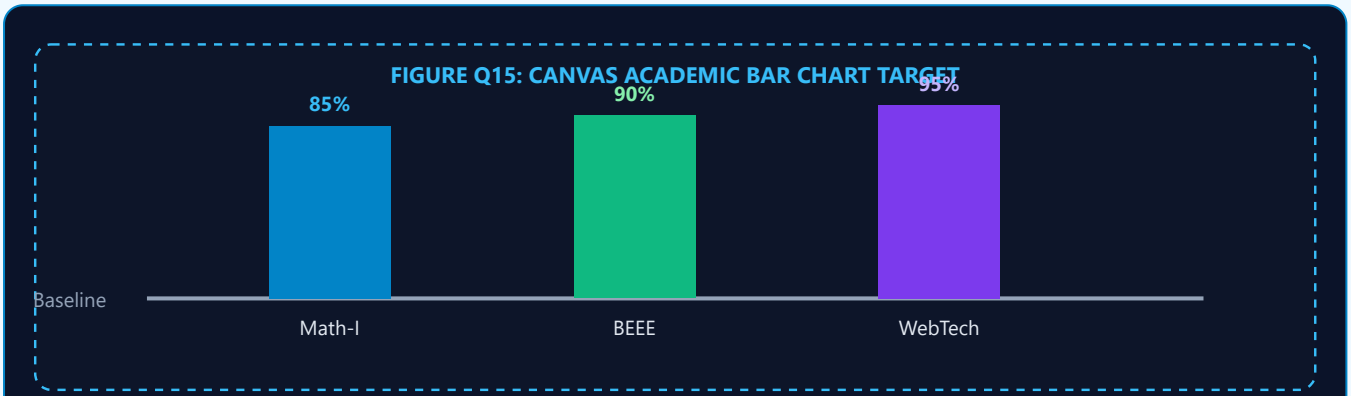


Fig Q15: Procedural Canvas Raster Visualization Target

Step-by-Step Solution:

```
<canvas id="chart" width="450" height="200" style="background:#f8fafc; border:1px solid
#cbd5e1;"></canvas>
<script>
  const canvas = document.getElementById("chart");
  const ctx = canvas.getContext("2d");

  const subjects = [
    { name: "Math-I", marks: 85, color: "#0284c7" },
    { name: "BEEE", marks: 90, color: "#10b981" },
    { name: "WebTech", marks: 95, color: "#7c3aed" }
  ];

  const startX = 60;
  const barWidth = 60;
  const gap = 50;
  const baselineY = 150;

  // Draw Baseline Axis
  ctx.beginPath();
  ctx.moveTo(30, baselineY);
  ctx.lineTo(420, baselineY);
  ctx.strokeStyle = "#475569";
  ctx.lineWidth = 2;
  ctx.stroke();

  // Draw Bars & Labels
  subjects.forEach((sub, i) => {
    const x = startX + i * (barWidth + gap);
    const barHeight = sub.marks * 1.1;
    const y = baselineY - barHeight;

    // Bar Fill
    ctx.fillStyle = sub.color;
    ctx.fillRect(x, y, barWidth, barHeight);

    // Score Text
    ctx.fillStyle = "#0f172a";
    ctx.font = "bold 12px sans-serif";
    ctx.textAlign = "center";
    ctx.fillText(sub.marks + "%", x + barWidth / 2, y - 6);

    // Subject Name Text
    ctx.font = "600 11px sans-serif";
    ctx.fillStyle = "#334155";
    ctx.fillText(sub.name, x + barWidth / 2, baselineY + 18);
  });
</script>
```

🎯 Complete script renders baseline axis, scaled dynamic bars, and dual centered text annotations.